DEZSYS GK7.1

Flood Monitor REST

Hochwasser-Frühwarnsystem mit Spring Boot, REST, JSON und XML

Inhaltsverzeichnis

1. Projekt einrichten 3
   1.1 Startanleitung 3
   1.2 Konfiguration 4
2. Messwerte simulieren 5
   2.1 Regeln 5
   2.2 Beispiel aus der Konsole 5
3. Warnstufe berechnen 6
   3.1 Beispiel aus der Konsole 6
4. REST-Schnittstelle implementieren 7
   4.1 Endpunkte 7
   4.2 Filter testen 7
   4.3 Beispiel Statistik 8
5. JSON und XML: Content Negotiation 9
   5.1 Beispiele für Anfragen 9
6. HTTP-Statuscodes und Fehlerbehandlung 10
   6.1 Beispiel 10
7. Webclient 11
8. Architektur und Datenmodell 14
   8.1 Schichten 14
   8.2 Datenmodell 14
   8.3 Ablauf einer Anfrage 15
   8.4 Projektstruktur 15
9. Automatisierte Tests 16
10. API-Dokumentation 18
   10.1 OpenAPI 18
   10.2 Authentifizierung 18
   10.3 Swagger UI 19
   10.4 OpenAPI-Datei im Repository 20
11. Technische Mindestanforderungen 21
12. Fragestellungen 22
13. Reflexion 24
14. Verwendete AI-Prompts 25

1. Projekt einrichten

Das Projekt basiert auf dem Starter-Projekt vom Lehrer und ist ein Spring-Boot-Projekt mit Gradle. Neben Spring Web wurde Jackson XML für die XML-Ausgabe eingebunden.

```
dependencies { Add Starters...
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'com.fasterxml.jackson.dataformat:jackson-dataformat-xml'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

Abbildung 1: Abhängigkeiten in build.gradle (Spring Web und Jackson XML)

1.1 Startanleitung

Man braucht Java 17. In IntelliJ wird Java 17 als Projekt-SDK und als Gradle JVM eingestellt. Für das Terminal muss die Umgebungsvariable JAVA_HOME auf Java 17 zeigen. Danach startet man im Projektordner:

.\gradlew.bat bootRun

Die Tests startet man mit:

.\gradlew.bat test

Wenn die Anwendung läuft, erreicht man den Webclient unter http://localhost:8080, die API unter http://localhost:8080/api/v1 und die Swagger-Seite unter http://localhost:8080/swagger-ui.html.

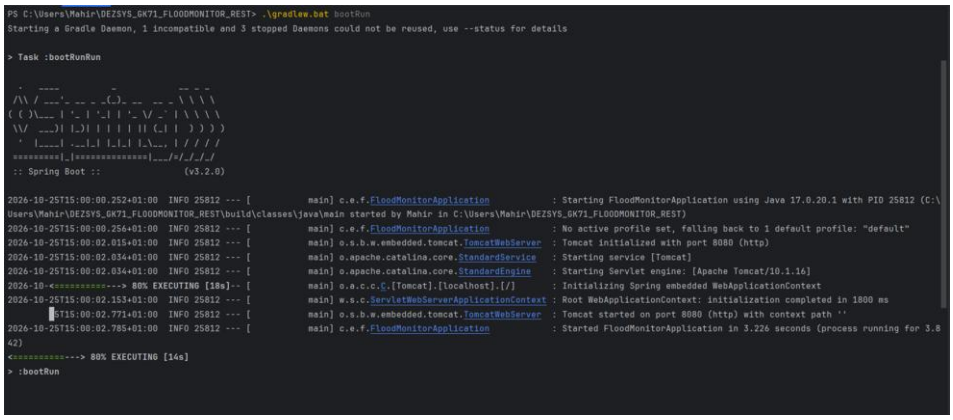


Abbildung 2: Start der Anwendung mit .\gradlew.bat bootRun

1.2 Konfiguration

Die Einstellungen stehen in src/main/resources/application.properties:

Einstellung	Bedeutung
server.port=8080	Port der Anwendung
simulation.interval=10000	alle 10 Sekunden eine neue Messung
simulation.enabled=true	Simulation ein- oder ausschalten
api.key=floodmonitor-demo-key	API-Key für die Anfragen (nur ein Beispiel-Key, kein echtes Passwort)

2. Messwerte simulieren

Die Simulation ist in der Klasse SimulationService. Die Methode simulate() wird durch @Scheduled automatisch alle 10 Sekunden aufgerufen (Einstellung simulation.interval). Mit simulation.enabled=false kann man die Simulation ausschalten, das wird bei den Tests verwendet.

Bei jedem Durchlauf bekommt jede aktive Station eine neue Messung. Dabei wird immer die vorherige Messung der Station verwendet, damit sich die Werte nur langsam ändern.

2.1 Regeln

Wert	Regel
Startwerte	Wasserstand = Normalwasserstand der Station, Regen = 0 mm/h, Temperatur = 12 °C, Akku zufällig zwischen 50 und 100 %
Niederschlag	ändert sich pro Messung um höchstens ±3 mm/h und liegt immer zwischen 0 und 30 mm/h
Wasserstand	neuer Wert = alter Wert + Regen × 0,8 − (alter Wert − Normalwert) × 0,05 + Zufall (±2 cm). Bei Regen steigt der Wasserstand, ohne Regen sinkt er langsam zurück zum Normalwert. Er kann nie negativ werden.
Durchflussmenge	Wasserstand × 2,5, also mehr Wasser bedeutet mehr Durchfluss
Temperatur	ändert sich pro Messung um höchstens ±0,3 °C
Akkustand	sinkt pro Messung um 0 oder 1 %. Unter 15 % wechselt die Station auf MAINTENANCE. Bei 5 % wird der Akku getauscht (wieder 100 %) und die Station ist wieder ONLINE.
Inaktive Stationen	Für Stationen mit isActive = false oder Status OFFLINE werden keine Messungen erzeugt (z. B. ST-004 Graz).

2.2 Beispiel aus der Konsole

Man sieht, dass der Wasserstand bei Regen langsam steigt und der Durchfluss mitwächst. Für ST-004 kommen keine Messungen, weil die Station deaktiviert ist.

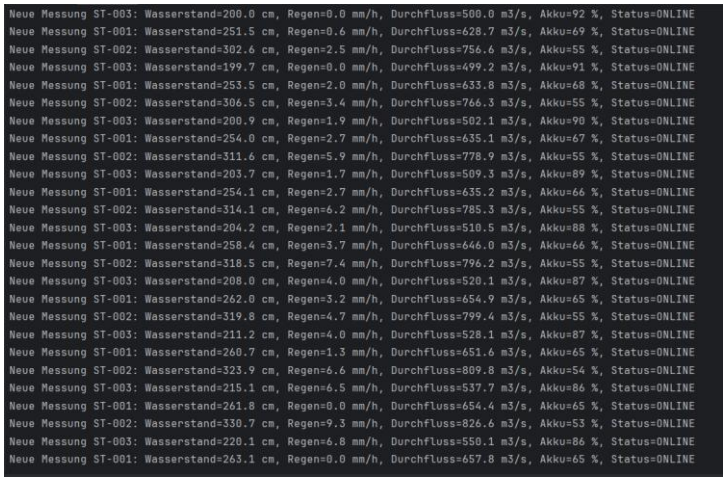


Abbildung 3: Konsolenausgabe der Simulation

3. Warnstufe berechnen

Die Warnstufe wird nur am Server berechnet (Methode calculateWarningLevel im StationService). Der Client kann sie nicht vorgeben. Sie wird bei jeder neuen Messung berechnet und in der Messung gespeichert.

Bedingung	Warnstufe
Wasserstand kleiner als der Warnwert	NORMAL
Wasserstand ab dem Warnwert	WARNING
Wasserstand ab dem kritischen Wert	CRITICAL
keine Messung, negativer Wasserstand oder letzte Messung älter als 3 Intervalle (30 Sekunden)	UNKNOWN

Die Grenzwerte sind bei jeder Station anders (siehe Kapitel 8.2). Niederschlag und Anstiegsgeschwindigkeit werden nicht verwendet, weil das laut Aufgabe optional ist.

3.1 Beispiel aus der Konsole

Zum Testen wurden die Grenzwerte von ST-001 kurz auf 255 cm (Warnung) und 260 cm (kritisch) gesetzt. Man sieht, wie die Warnstufe wechselt:

ST-001: Wasserstand=253.3 cm, Warnstufe=NORMAL

ST-001: Wasserstand=256.1 cm, Warnstufe=WARNING

ST-001: Wasserstand=260.4 cm, Warnstufe=CRITICAL

4. REST-Schnittstelle implementieren

Die Basis-URL ist /api/v1. Alle Endpunkte sind im StationController, die Logik (Filter, Statistik, Warnungen) ist im StationService.

4.1 Endpunkte

Methode	Endpunkt	Beschreibung	Filter
GET	/api/v1/stations	alle Stationen	river, isActive, warningLevel
GET	/api/v1/stations/{stationId}	eine Station	–
GET	/api/v1/stations/{stationId}/measurements/latest	aktuellste Messung	–
GET	/api/v1/stations/{stationId}/measurements	historische Messungen, neueste zuerst	from, to, limit (Standard 100), warningLevel
GET	/api/v1/stations/{stationId}/statistics	Statistik	from, to
GET	/api/v1/alerts	aktuelle Warnungen	warningLevel, river

Gibt es die Station, aber noch keine Messung (z. B. ST-004), liefert /measurements/latest den Statuscode 204 No Content. Der Endpunkt /alerts liefert die letzte Messung von allen Stationen, die gerade WARNING oder CRITICAL haben.

Die Statistik enthält den niedrigsten, höchsten und durchschnittlichen Wasserstand, die durchschnittliche Durchflussmenge, die Summe des Niederschlags, die Anzahl der Messungen, die Anzahl der Warnungen (WARNING und CRITICAL) und die Anzahl der kritischen Warnungen.

4.2 Filter testen

Anfrage	Ergebnis
/api/v1/stations?river=Donau	Wien und Linz
/api/v1/stations?isActive=false	nur Graz
/api/v1/stations?warningLevel=UNKNOWN	nur Graz (hat keine Messungen)
/api/v1/stations/ST-001/measurements?limit=3	die 3 neuesten Messungen von ST-001
/api/v1/stations/ST-004/measurements/latest	204 No Content
/api/v1/alerts	leere Liste, wenn alle Stationen NORMAL sind

4.3 Beispiel Statistik

GET /api/v1/stations/ST-001/statistics
{"stationId":"ST-001", "minWaterLevel":250.0, "maxWaterLevel":306.6,
"averageWaterLevel":276.8, "averageFlowRate":692.0, "totalRainfall":106.8,
"measurementCount":28, "warningCount":0, "criticalCount":0}

5. JSON und XML: Content Negotiation

Für XML wurde jackson-dataformat-xml in build.gradle eingebunden. Spring Boot erkennt das automatisch und kann damit jede Antwort als JSON oder als XML ausgeben. Welches Format zurückkommt, entscheidet der Accept-Header der Anfrage:

Accept-Header	Antwort
application/json	JSON
application/xml	XML
kein Accept-Header	JSON (Standard)
nicht unterstützt, z. B. text/plain	406 Not Acceptable

Damit die XML-Elemente klein geschrieben werden (<measurement> statt <Measurement>), steht über den Model-Klassen die Annotation @JacksonXmlRootElement. Die Zeitstempel werden ohne Nanosekunden gespeichert, damit sie wie in der Aufgabe aussehen (2026-10-25T15:42:40Z).

5.1 Beispiele für Anfragen

JSON-Anfrage:

curl.exe -H "X-API-KEY: floodmonitor-demo-key" -H "Accept: application/json" http://localhost:8080/api/v1/stations/ST-001/measurements/latest

{"stationId":"ST-001", "timestamp":"2026-10-25T15:42:40Z", "waterLevel":286.3, "flowRate":715.7, "rainfall":4.3, "temperature":12.1, "batteryLevel":61, "status":"ONLINE", "warningLevel":"NORMAL"}

XML-Anfrage:

curl.exe -H "X-API-KEY: floodmonitor-demo-key" -H "Accept: application/xml" http://localhost:8080/api/v1/stations/ST-001/measurements/latest

<measurement><stationId>ST-001</stationId><timestamp>2026-10-25T15:42:40Z</timestamp><waterLevel>286.3</waterLevel><flowRate>715.7</flowRate><rainfall>4.3</rainfall><temperature>12.1</temperature><batteryLevel>61</batteryLevel><status>ONLINE</status><warningLevel>NORMAL</warningLevel></measurement>

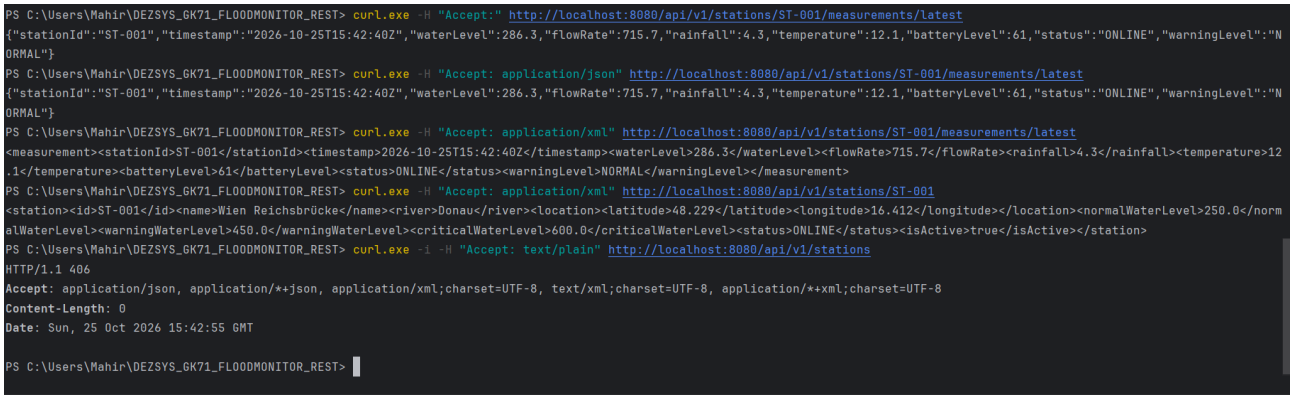


Abbildung 4: JSON- und XML-Antwort derselben Messung

Man sieht, dass JSON und XML genau dieselben Werte enthalten.

6. HTTP-Statuscodes und Fehlerbehandlung

Alle Fehler werden zentral in der Klasse GlobalExceptionHandler behandelt (Annotation @RestControllerAdvice). Jede Fehlerantwort hat denselben Aufbau (Klasse ErrorResponse) und kommt je nach Accept-Header als JSON oder XML. Stacktraces werden nie an den Client geschickt.

Situation	Beispiel	Statuscode
unbekannte Station	/stations/ST-999	404
unbekannter Pfad	/api/v1/xyz	404
ungültige Warnstufe	?warningLevel=HOCH	400
ungültiges Datumsformat	?from=gestern	400
from liegt nach to	?from=...12:00Z&to=...08:00Z	400
limit kleiner als 1 oder größer als 1000	?limit=-5, ?limit=5000	400
API-Key fehlt	ohne Header X-API-KEY	401
API-Key ist falsch	X-API-KEY: falsch	403
Methode nicht erlaubt	POST /stations	405
Format nicht unterstützt	Accept: text/plain	406
Medientyp nicht unterstützt	–	415
sonstiger Fehler	–	500

Bei 406 kommt die Fehlermeldung immer als JSON, weil der Client ja kein unterstütztes Format angegeben hat.

6.1 Beispiel

{"timestamp":"2026-10-25T16:19:42Z", "status":404, "error":"Not Found", "message":"Station ST-999 wurde nicht gefunden", "path":"/api/v1/stations/ST-999"}



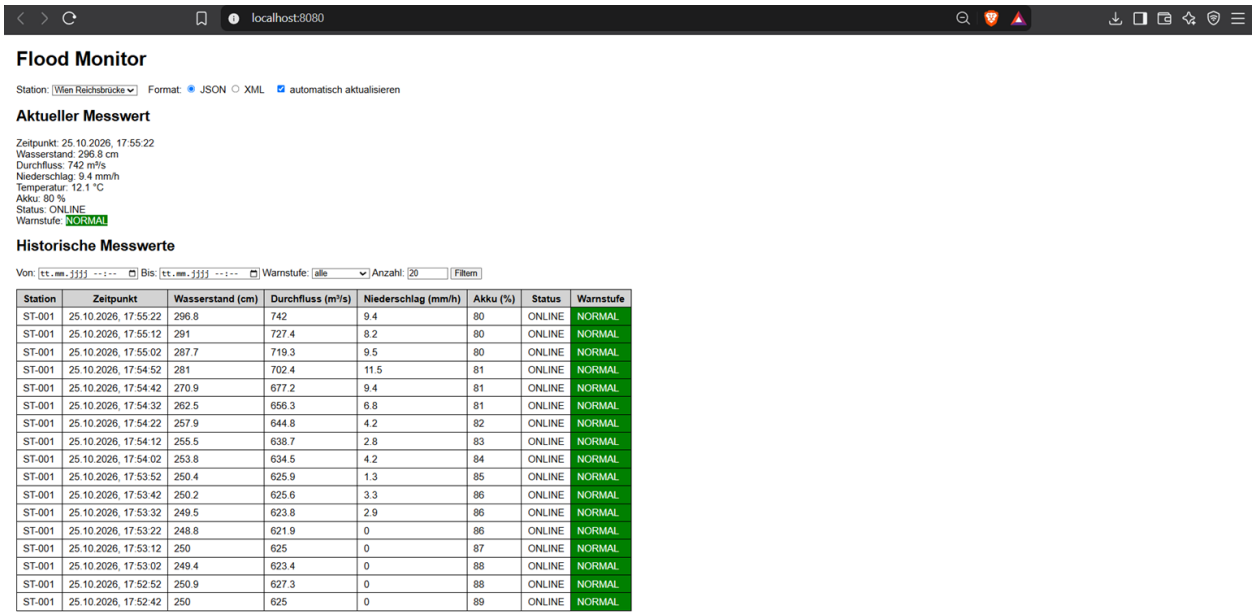
Abbildung 5: Derselbe 404-Fehler einmal als JSON und einmal als XML

7. Webclient

Der Webclient besteht aus index.html, style.css und app.js im Ordner src/main/resources/static und ist unter http://localhost:8080 erreichbar.

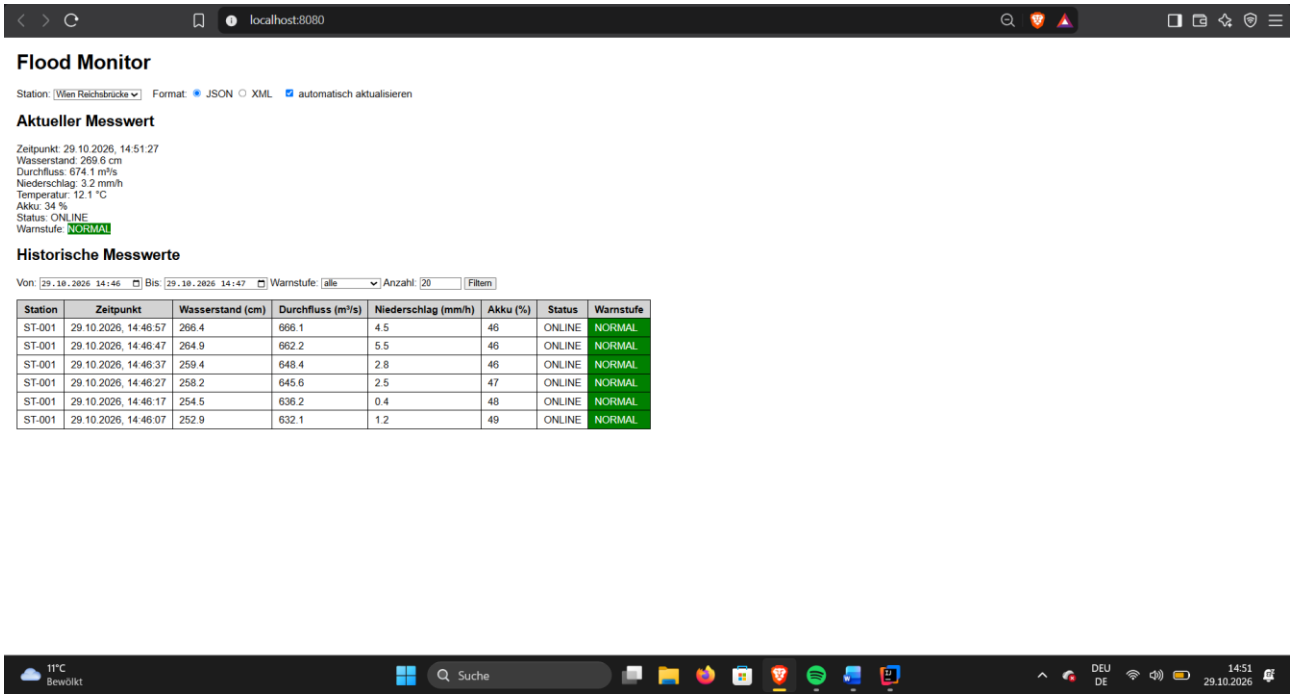
Man wählt eine Station aus und sieht den aktuellen Messwert und eine Tabelle mit den historischen Messwerten, die man nach Zeitraum, Warnstufe und Anzahl filtern kann. Mit den Buttons schaltet man zwischen JSON und XML um, bei XML liest jQuery die Werte aus dem XML aus.

Die Daten werden alle 10 Sekunden neu geladen. Während einer Anfrage steht „Daten werden geladen…“, bei einem Fehler kommt eine rote Fehlermeldung. Die Warnstufen sind farbig: NORMAL grün, WARNING orange, CRITICAL rot und UNKNOWN grau.



Station	Zeitpunkt	Wasserstand (cm)	Durchfluss (m³/s)	Niederschlag (mm/h)	Akku (%)	Status	Warnstufe
ST-001	25.10.2026, 17:55:22	296.8	742	9.4	80	ONLINE	NORMAL
ST-001	25.10.2026, 17:55:12	291	727.4	8.2	80	ONLINE	NORMAL
ST-001	25.10.2026, 17:55:02	287.7	719.3	9.5	80	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:52	281	702.4	11.5	81	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:42	270.9	677.2	9.4	81	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:32	262.5	656.3	6.8	81	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:22	257.9	644.8	4.2	82	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:12	255.5	638.7	2.8	83	ONLINE	NORMAL
ST-001	25.10.2026, 17:54:02	253.8	634.5	4.2	84	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:52	250.4	625.9	1.3	85	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:42	250.2	625.6	3.3	86	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:32	249.5	623.8	2.9	86	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:22	248.8	621.9	0	86	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:12	250	625	0	87	ONLINE	NORMAL
ST-001	25.10.2026, 17:53:02	249.4	623.4	0	88	ONLINE	NORMAL
ST-001	25.10.2026, 17:52:52	250.9	627.3	0	88	ONLINE	NORMAL
ST-001	25.10.2026, 17:52:42	250	625	0	89	ONLINE	NORMAL

Abbildung 6: Webclient im JSON-Modus mit aktuellem Messwert und Tabelle



Station	Zeitpunkt	Wasserstand (cm)	Durchfluss (m³/s)	Niederschlag (mm/h)	Akku (%)	Status	Warnstufe
ST-001	29.10.2026, 14:46:57	266.4	666.1	4.5	46	ONLINE	NORMAL
ST-001	29.10.2026, 14:46:47	264.9	662.2	5.5	46	ONLINE	NORMAL
ST-001	29.10.2026, 14:46:37	259.4	648.4	2.8	46	ONLINE	NORMAL
ST-001	29.10.2026, 14:46:27	258.2	645.6	2.5	47	ONLINE	NORMAL
ST-001	29.10.2026, 14:46:17	254.5	636.2	0.4	48	ONLINE	NORMAL
ST-001	29.10.2026, 14:46:07	252.9	632.1	1.2	49	ONLINE	NORMAL

Abbildung 7: Filter nach Zeitraum (14:46 bis 14:47)

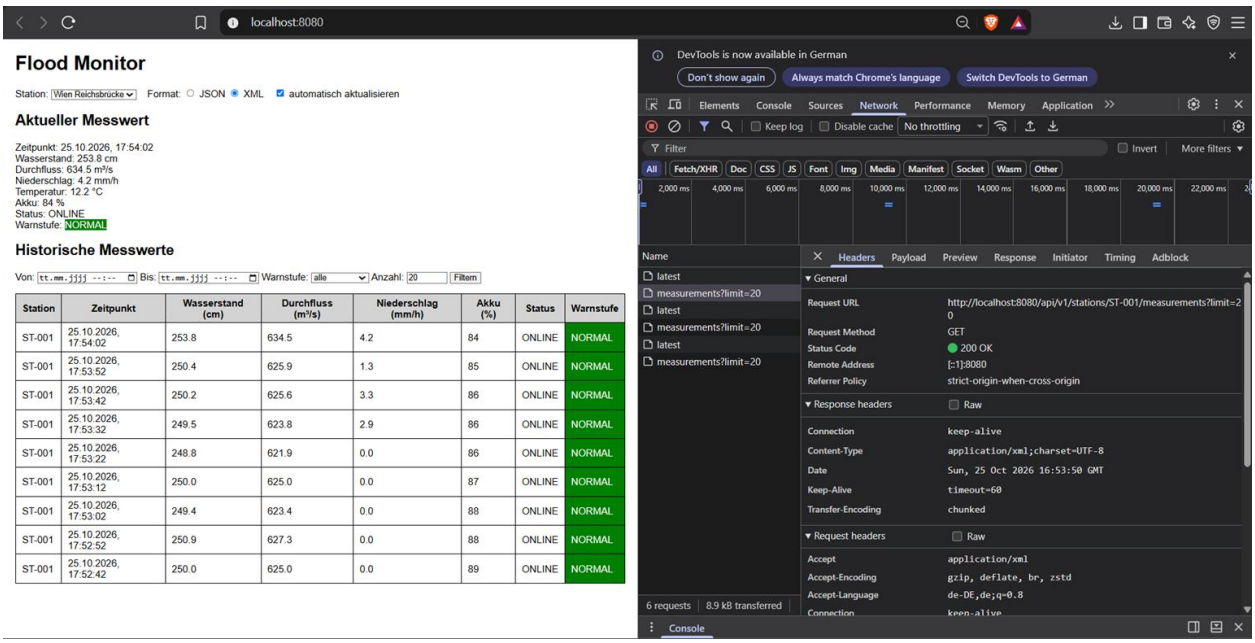


Abbildung 8: Webclient im XML-Modus, in den Entwicklertools sieht man Accept: application/xml

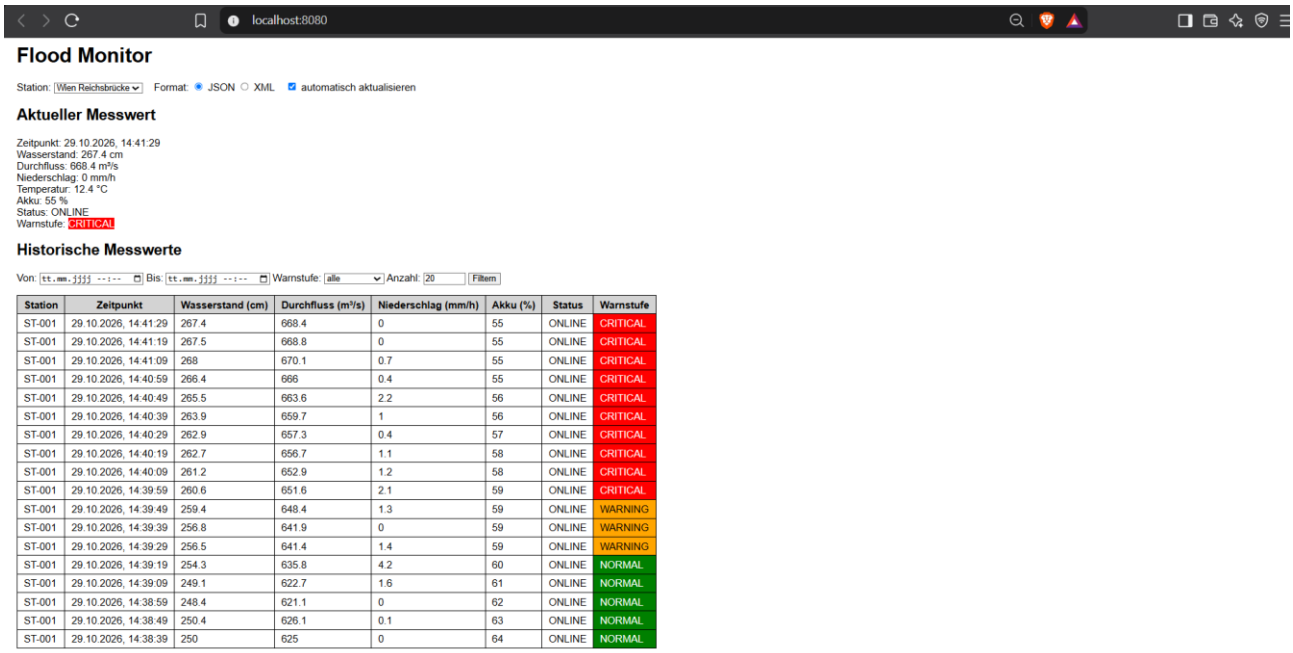


Abbildung 9: Farben der Warnstufen: NORMAL grün, WARNING orange, CRITICAL rot

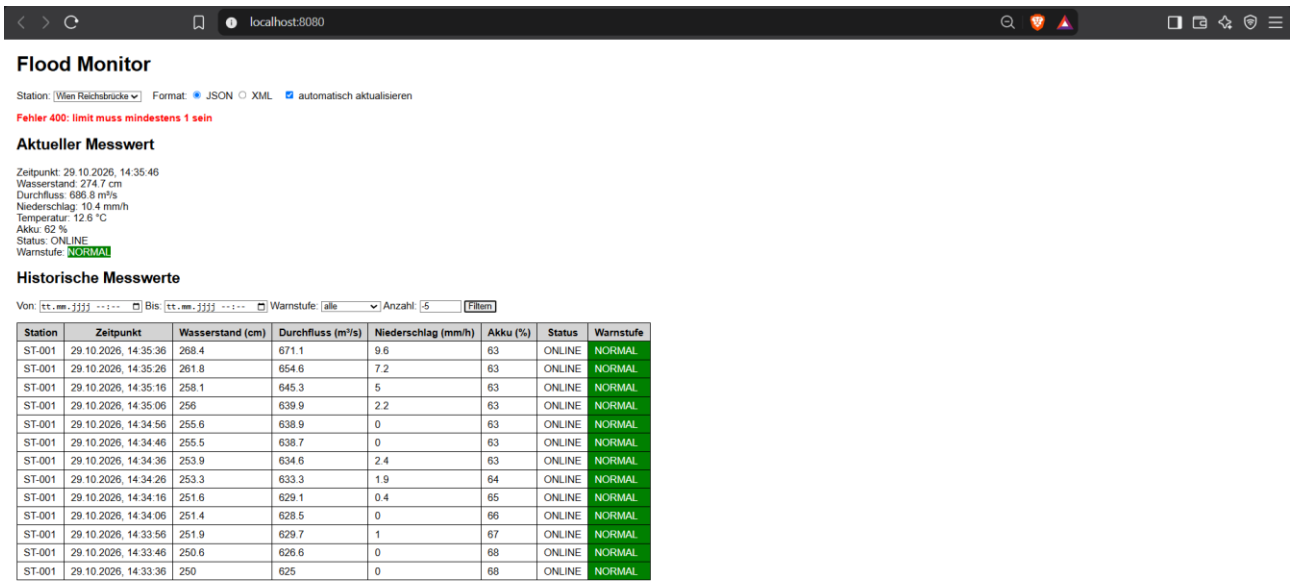


Abbildung 10: Fehlermeldung bei ungültiger Anzahl (limit = -5)

8. Architektur und Datenmodell

8.1 Schichten

Die Anwendung ist in Schichten aufgeteilt, jede Schicht hat ein eigenes Package. Der Controller nimmt nur die Anfrage entgegen und ruft den Service auf, die eigentliche Logik ist im Service.

Bereich	Klassen	Aufgabe
Model	Station, Location, Measurement, Statistics, WarningLevel, StationStatus	Datenklassen und Enums
Repository	StationRepository, MeasurementRepository	speichert Stationen und Messungen im Arbeitsspeicher (keine Datenbank)
Service	StationService	Filter, Warnstufe, Statistik, Warnungen, Prüfung der Eingaben
	SimulationService	erzeugt regelmäßig neue Messwerte
Controller	StationController	nimmt HTTP-Anfragen entgegen und gibt die Antwort zurück
Exception Handling	GlobalExceptionHandler, ErrorResponse, StationNotFoundException	einheitliche Fehlerantworten
Security	ApiKeyInterceptor, WebConfig	prüft den API-Key
Client	index.html, style.css, app.js	Webclient zum Anzeigen der Daten

8.2 Datenmodell

Klasse	Inhalt
Station	id, name, river, location, normalWaterLevel, warningWaterLevel, criticalWaterLevel, isActive, status
Location	latitude (Breitengrad), longitude (Längengrad)
Measurement	stationId, timestamp, waterLevel (cm), flowRate (m^3/s), rainfall (mm/h), temperature (°C), batteryLevel (%), status, warningLevel
Statistics	die Werte der Statistik (siehe Kapitel 4.1)
WarningLevel (Enum)	NORMAL, WARNING, CRITICAL, UNKNOWN
StationStatus (Enum)	ONLINE, MAINTENANCE, OFFLINE

Zeitpunkte werden als Instant aus java.time gespeichert. Es gibt vier Stationen:

ID	Name	Fluss	Normal	Warnung	Kritisch	Aktiv
ST-001	Wien Reichsbrücke	Donau	250 cm	450 cm	600 cm	ja
ST-002	Linz	Donau	300 cm	550 cm	700 cm	ja
ST-003	Salzburg	Salzach	200 cm	450 cm	600 cm	ja

ID	Name	Fluss	Normal	Warnung	Kritisch	Aktiv
ST-004	Graz	Mur	150 cm	300 cm	400 cm	nein

Graz ist absichtlich deaktiviert, damit man sieht, dass für inaktive Stationen keine Messungen erzeugt werden.

8.3 Ablauf einer Anfrage

Am Beispiel GET /api/v1/stations/ST-001/measurements:

1. Der Webclient oder curl schickt die Anfrage mit Accept-Header und API-Key.
2. Der ApiKeyInterceptor prüft den API-Key.
3. Der StationController nimmt die Parameter entgegen.
4. Der StationService prüft die Eingaben und filtert die Messwerte.
5. Das MeasurementRepository liefert die gespeicherten Messwerte.
6. Die Antwort geht als JSON oder XML zurück.

Bei einem Fehler, z. B. einer unbekannten Station, schickt der GlobalExceptionHandler eine einheitliche Fehlerantwort. Nebenbei speichert der SimulationService alle 10 Sekunden neue Messungen im MeasurementRepository.

8.4 Projektstruktur

Ordner	Dateien
src/main/java/.../floodmonitor	FloodMonitorApplication.java
controller	StationController.java
exception	ErrorResponse.java, GlobalExceptionHandler.java, StationNotFoundException.java
model	Location.java, Measurement.java, Station.java, StationStatus.java, Statistics.java, WarningLevel.java
repository	MeasurementRepository.java, StationRepository.java
security	ApiKeyInterceptor.java, WebConfig.java
service	SimulationService.java, StationService.java
src/main/resources	application.properties
src/main/resources/static	index.html, style.css, app.js
src/test/java/.../floodmonitor	FloodMonitorApiTest.java
docs	openapi.yaml

9. Automatisierte Tests

Die Tests sind in der Klasse FloodMonitorApiTest (src/test/java) und werden mit .\gradlew.bat test gestartet.

Damit die Tests nicht zufällig fehlschlagen, ist die Simulation ausgeschaltet (simulation.enabled=false) und die Messwerte werden mit festen Werten angelegt. Vor jedem Test werden alle Messungen gelöscht (@BeforeEach). Die Anfragen laufen über MockMvc, der Server muss dafür nicht laufen.

Nr.	Test	Was wird geprüft
1	getAllStationsAsJson	alle Stationen als JSON, Status 200
2	getAllStationsAsXml	alle Stationen als XML, Status 200
3	getExistingStation	Station ST-001 wird gefunden
4	getUnknownStation	ST-999 liefert 404 mit Fehlermeldung
5	getLatestMeasurement	die neueste Messung wird zurückgegeben
6	filterByTimeRange	nur Messungen im Zeitraum werden geliefert
7	invalidTimeRange	from nach to liefert 400
8	warningLevelNormal	300 cm → NORMAL
9	warningLevelWarning	450 cm und 500 cm → WARNING
10	warningLevelCritical	600 cm und 700 cm → CRITICAL
11	unsupportedFormat	Accept: text/plain liefert 406
12	maxLimit	limit=1000 erlaubt, limit=1001 liefert 400
13	missingApiKey	ohne API-Key kommt 401
14	wrongApiKey	falscher API-Key liefert 403

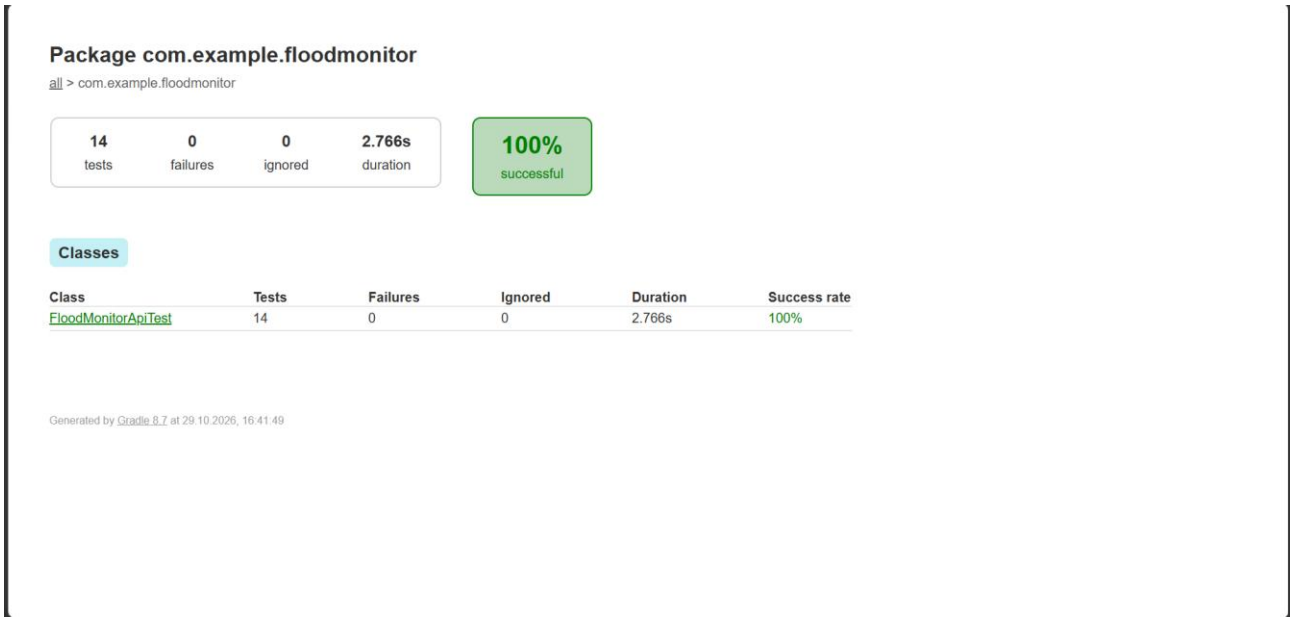
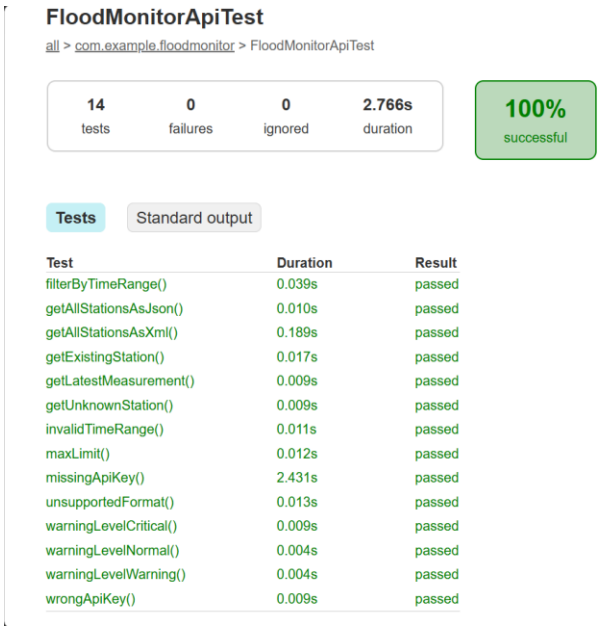


Abbildung 11: Gradle-Testbericht: Übersicht



FloodMonitorApiTest

all > com.example.floodmonitor > FloodMonitorApiTest

14	0	0	2.766s
tests	failures	ignored	duration

100% successful

Tests | Standard output

Test	Duration	Result
filterByTimeRange()	0.039s	passed
getAllStationsAsJson()	0.010s	passed
getAllStationsAsXml()	0.189s	passed
getExistingStation()	0.017s	passed
getLatestMeasurement()	0.009s	passed
getUnknownStation()	0.009s	passed
invalidTimeRange()	0.011s	passed
maxLimit()	0.012s	passed
missingApiKey()	2.431s	passed
unsupportedFormat()	0.013s	passed
warningLevelCritical()	0.009s	passed
warningLevelNormal()	0.004s	passed
warningLevelWarning()	0.004s	passed
wrongApiKey()	0.009s	passed

Abbildung 12: Gradle-Testbericht: alle Tests einzeln mit Ergebnis

10. API-Dokumentation

10.1 OpenAPI

Die API ist mit OpenAPI dokumentiert, einem Standard, mit dem man eine REST-API in einer YAML- oder JSON-Datei beschreibt. Die Beschreibung erzeugt springdoc-openapi automatisch aus dem Code. Mit Annotationen wie @Operation, @Parameter und @ApiResponse im StationController wurden Beschreibungen, Statuscodes und die Authentifizierung ergänzt.

Was	Wo
Swagger UI (Ansehen und Ausprobieren)	http://localhost:8080/swagger-ui.html
OpenAPI-Beschreibung als JSON	http://localhost:8080/v3/api-docs
OpenAPI-Datei im Repository	docs/openapi.yaml

Verlangt	Umgesetzt
Endpunkte und Methoden	alle 6 GET-Endpunkte mit Kurzbeschreibung
Parameter	Pfad- und Query-Parameter mit Beschreibung, Typ und Beispiel
Request-Bodies	keine, weil die API nur GET-Anfragen hat
Response-Bodies	Schemas Station, Location, Measurement, Statistics, ErrorResponse
Formate	application/json und application/xml
Statuscodes	200, 204, 400, 401, 403, 404, 406 je nach Endpunkt
Fehlerantworten	Schema ErrorResponse bei allen Fehlercodes
Authentifizierung	API-Key im Header X-API-KEY (Button „Authorize“)

10.2 Authentifizierung

Alle Endpunkte unter /api/ sind mit einem API-Key geschützt, der bei jeder Anfrage im Header X-API-KEY mitgeschickt werden muss. Der Key steht in application.properties und ist nur ein Beispiel-Key für die Abgabe.

Geprüft wird der Key vom ApiKeyInterceptor, der in der WebConfig für alle Pfade unter /api/** angemeldet ist. Fehlt der Key, kommt 401 Unauthorized, ist er falsch, kommt 403 Forbidden. Der Webclient schickt den Key automatisch mit. Da er dadurch im JavaScript steht, wäre das für ein echtes System nicht sicher genug.

Beispiel:
curl.exe -H "X-API-KEY: floodmonitor-demo-key" http://localhost:8080/api/v1/stations

10.3 Swagger UI

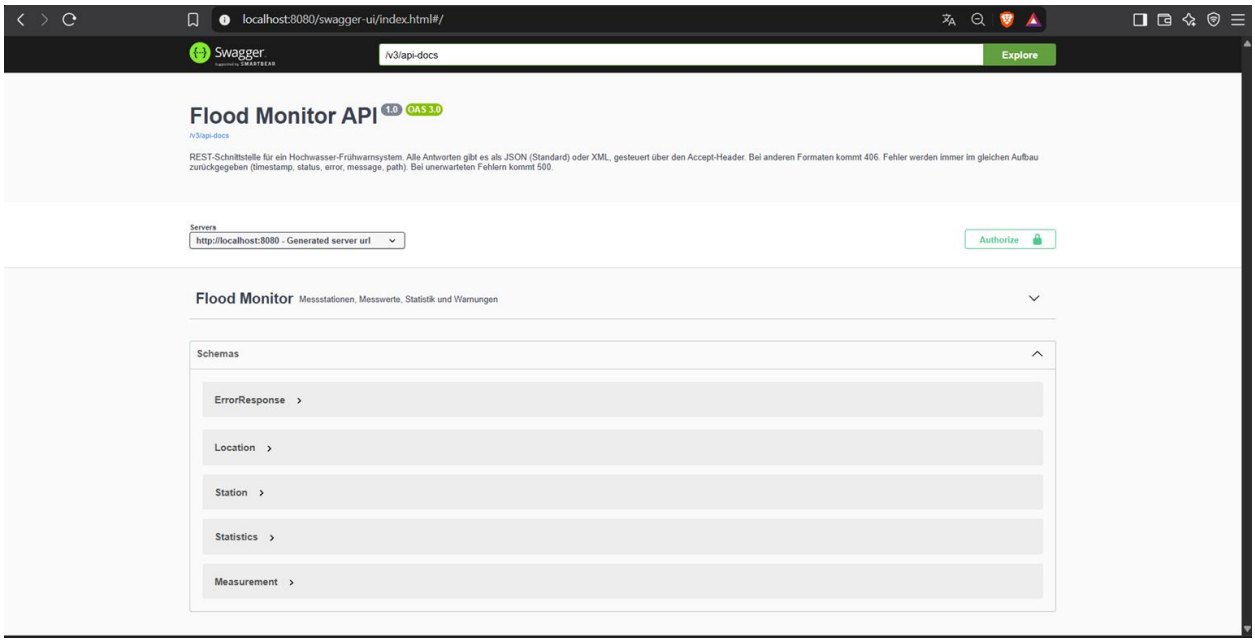


Abbildung 13: Swagger UI mit Titel, Beschreibung, Authorize-Button und Schemas

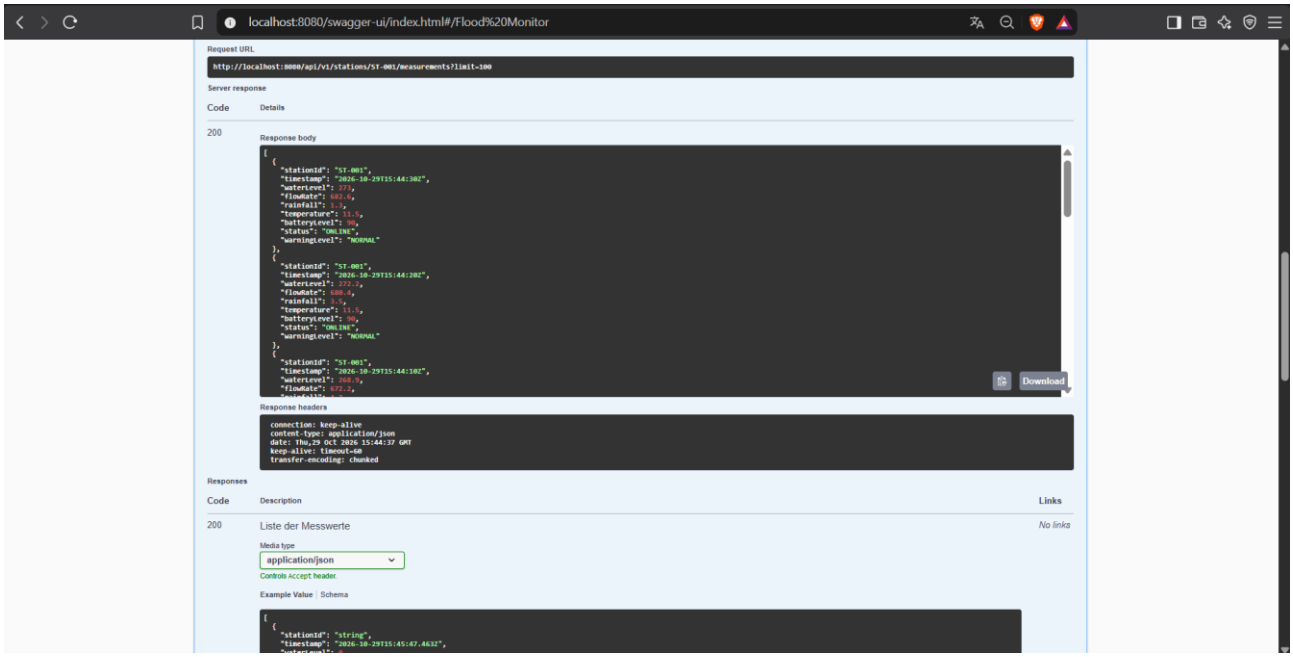


Abbildung 14: Ausprobieren eines Endpunkts mit „Try it out“, Antwort 200

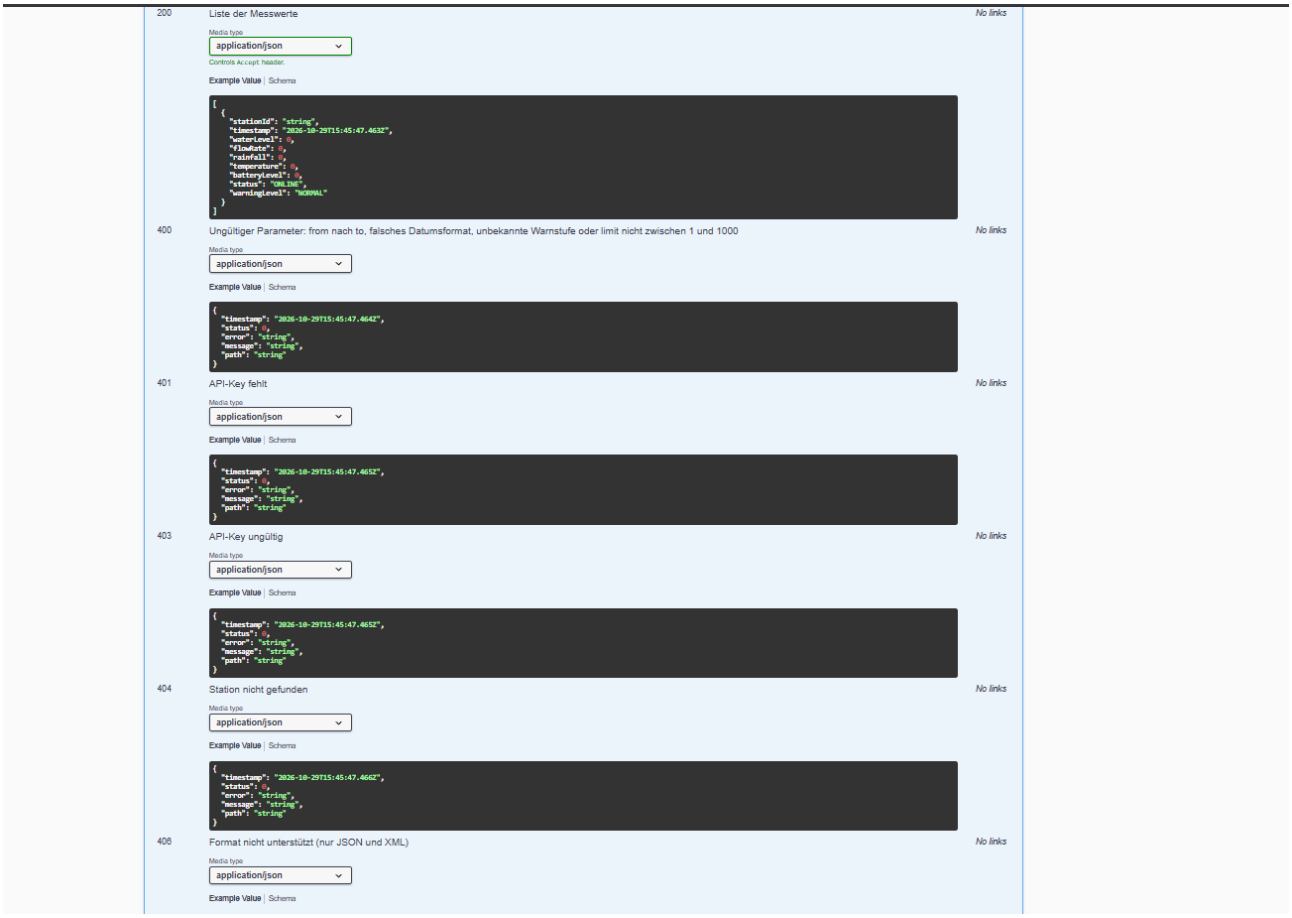


Abbildung 15: Dokumentierte Statuscodes und Fehlerantworten

10.4 OpenAPI-Datei im Repository

Die OpenAPI-Beschreibung wurde zusätzlich als Datei docs/openapi.yaml ins Repository gespeichert:

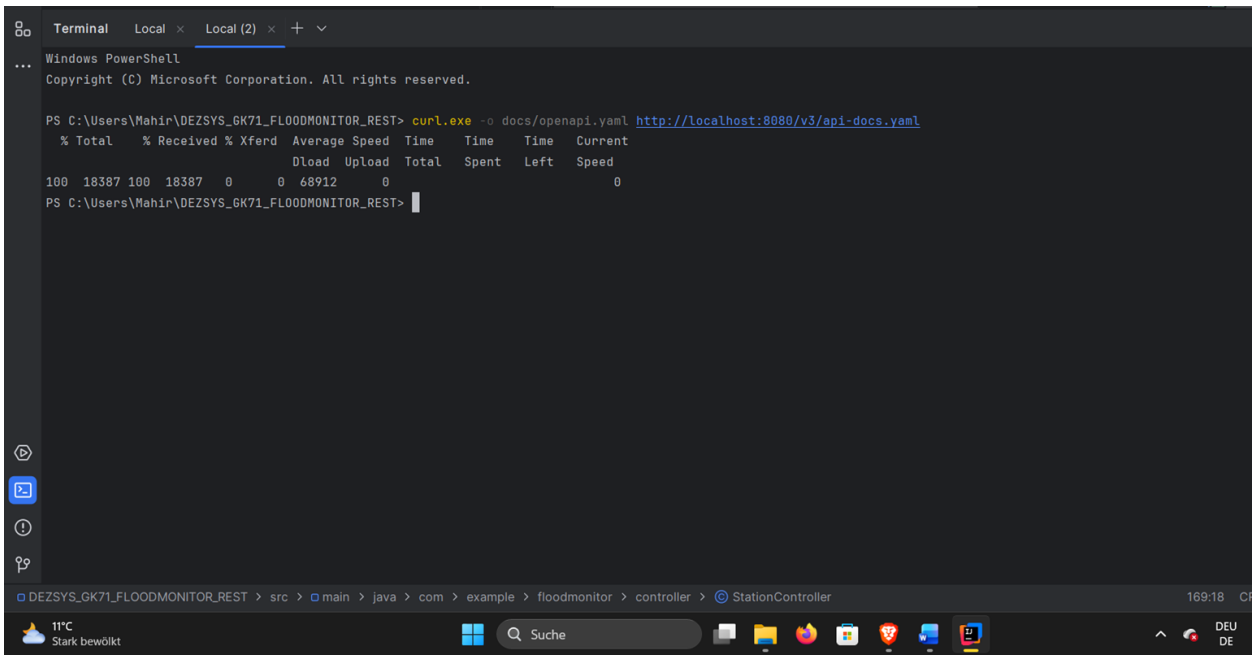


Abbildung 16: Speichern der OpenAPI-Datei mit curl

11. Technische Mindestanforderungen

Anforderung	Umsetzung
Java und Spring Boot	Java 17, Spring Boot 3.2.0
Maven oder Gradle	Gradle mit Gradle Wrapper (gradlew.bat)
REST, JSON und XML	REST-API unter /api/v1, JSON mit Jackson, XML mit jackson-dataformat-xml
HTML, CSS, JavaScript und jQuery	Webclient im Ordner static, jQuery 3.7.1
Git	GitHub-Repository mit einem Commit pro Arbeitsschritt
automatisierte Tests	14 Tests mit JUnit 5 und MockMvc

12. Fragestellungen

1. Was sind die Hauptmerkmale von JSON?

JSON ist ein Textformat zum Datenaustausch. Daten werden als Schlüssel-Wert-Paare in Objekten und Listen gespeichert, unterstützt werden Strings, Zahlen, Booleans und null. Es ist leicht lesbar, kompakter als XML und unabhängig von der Programmiersprache.

Quelle: https://www.json.org/json-en.html

2. Was sind die Hauptmerkmale von XML?

XML (Extensible Markup Language) ist ein Textformat vom W3C, bei dem Daten zwischen Start- und End-Tags stehen. Man kann eigene Tags definieren, die Daten sind als Baum aufgebaut und Elemente können Attribute haben. Ein XML-Dokument muss wohlgeformt sein und kann mit einem Schema geprüft werden. Es braucht mehr Platz als JSON.

Quelle: https://www.w3.org/XML/

3. Wie unterscheidet die Applikation, ob JSON oder XML zurückgegeben wird?

Über den Accept-Header der Anfrage (Content Negotiation). Spring Boot wählt je nach Accept: application/json oder Accept: application/xml den passenden Converter (Jackson bzw. Jackson XML). Ohne Accept-Header kommt JSON, bei einem nicht unterstützten Format antwortet der Server mit 406.

Quelle: https://developer.mozilla.org/en-US/docs/Web/HTTP/Content_negotiation

4. Was zeichnet ein REST-Interface aus und warum wird es so oft verwendet?

Bei REST hat jede Ressource eine eigene URL und wird über die HTTP-Methoden angesprochen. Die Kommunikation ist zustandslos, Client und Server sind getrennt und die Daten werden z. B. als JSON oder XML übertragen. REST ist so verbreitet, weil es auf HTTP aufbaut, das jede Sprache und jedes Gerät schon kann, und weil es einfach, leichtgewichtig und gut skalierbar ist.

Quelle: https://www.ibm.com/topics/rest-apis

5. Was ist der Unterschied zwischen GET, POST, PUT und PATCH?

GET liest Daten und ändert nichts am Server. POST erstellt eine neue Ressource. PUT ersetzt eine bestehende Ressource komplett, PATCH ändert nur einzelne Felder davon. GET und PUT sind idempotent, das heißt, mehrmaliges Ausführen hat dieselbe Wirkung wie einmal, bei POST entsteht jedes Mal eine neue Ressource.

Quelle: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

6. Welche Stellen im Code muss ich für eine neue Methode / einen neuen Pfad ändern?

Im Controller kommt eine neue Methode mit @GetMapping, @PostMapping usw. und dem Pfad dazu, die Logik dafür gehört in den Service. Falls neue Daten gespeichert werden, muss man auch Repository und Model anpassen. Außerdem sollte man die OpenAPI-Annotationen, die Tests und die Fehlerbehandlung ergänzen.

Quelle: https://spring.io/guides/gs/rest-service

7. Kann man OpenAPI und Swagger vergleichen?

Ja, aber sie sind nicht dasselbe. OpenAPI ist die Spezifikation, wie man eine REST-API in einer YAML- oder JSON-Datei beschreibt. Swagger sind Tools von SmartBear, die mit dieser Spezifikation arbeiten, z. B. Swagger UI oder Swagger Editor. Früher hieß die Spezifikation selbst Swagger, 2015 wurde sie in OpenAPI umbenannt. In meinem Projekt erzeugt springdoc die OpenAPI-Beschreibung und Swagger UI zeigt sie an.

Quelle: https://nordicapis.com/whats-the-difference-between-swagger-and-openapi/ und REST-API und OpenAPI: Es handelt sich nicht um eine Entweder-Oder-Frage (YouTube): https://www.youtube.com/watch?v=pRS9LRBgjYg

8. Wo liegt der Fehler im Code-Snippet?

Es wird GET verwendet, um eine Station zu erstellen. GET darf aber nur Daten lesen, sonst wird z. B. bei jedem Neuladen oder Aufruf durch eine Suchmaschine eine neue Station angelegt. Richtig wäre @PostMapping("/stations") mit den Daten im Request-Body (@RequestBody) und der Antwort 201 Created. Es sollte kein Verb wie add in der URL stehen.

Quelle: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET

13. Reflexion

Java-Version

Auf meinem PC war nur Java 8 installiert, das Projekt braucht aber Java 17. IntelliJ hat dann automatisch Java 25 genommen, damit funktioniert Gradle 8.7 aber nicht (maximal Java 21). Ich habe in IntelliJ Java 17 als Projekt-SDK und Gradle JVM eingestellt. Im Terminal kam trotzdem ein Fehler, weil dort noch Java 8 verwendet wurde. Das habe ich mit der Umgebungsvariable JAVA_HOME gelöst.

Zu viele Klassen

Am Anfang hatte ich für jede Aufgabe einen eigenen Service (WarningService, AlertService usw.). Das war für so ein kleines Projekt unübersichtlich, deshalb habe ich die Logik im StationService zusammengefasst.

API-Key

Die Tests für 401 und 403 sind fehlgeschlagen, weil ich vergessen hatte, den Interceptor in einer WebConfig anzumelden. Ohne die Tests wäre mir das wahrscheinlich nicht aufgefallen. Danach hat der Webclient trotzdem noch 401 bekommen, weil der Browser die alte app.js aus dem Cache genommen hat. Mit Strg+F5 bzw. „Disable cache“ war das Problem weg.

PowerShell

In PowerShell ist curl ein anderer Befehl, man muss curl.exe schreiben. URLs mit & müssen in Anführungszeichen stehen, sonst versteht PowerShell sie falsch.

14. Verwendete AI-Prompts

Nr.	Prompt	Ergebnis
1	Fehlermeldung bei gradlew.bat bootRun im Terminal (“compatible with Java 8“)	JAVA_HOME auf Java 17 gesetzt, danach startet das Projekt im Terminal
2	Gradle-Fehlermeldung „incompatible Java 25.0.4 and Gradle 8.7“	Gradle JVM und Projekt-SDK in IntelliJ auf Java 17 umgestellt
3	Fehlermeldung der Tests missingApiKey und wrongApiKey („expected 401 but was 200“)	fehlende WebConfig gefunden, danach liefen alle 14 Tests
4	Das ist meine aktuelle Dokumentation. Mach sie schöner, nur mit meinen Bildern.	einheitliches Layout mit Deckblatt und Inhaltsverzeichnis, Bildunterschriften
5	Screenshot der Test-Klasse: “Ich bekomme diesen Fehler hier nicht weg”	Test-Klasse Fehler behoben
6	Mehrere Fehler im Model und service Package, von allein nicht geschafft auszubessern	Fehler behoben mit Erklärung verstanden